PCB Defect Detection Under Image Misalignment

Data Preparation Report | COEN 6314 Course Project | October 2026

Basma Kurany | Concordia University, Gina Cody School of Engineering and Computer Science

Summary

This report describes the DeepPCB data and the reproducible preparation pipeline for measuring the effect of image misalignment on defect detection. It covers the source audit, group-aware split, controlled shifts, label handling, checks, and model-ready exports. Preparation results are reported here; detector performance is not. Proposal V4 lists Fast R-CNN, YOLOv10-Nano, and SSD-MobileNet as detector options. This project selects YOLOv10-Nano for the implementation.

Dataset at a glance

Measure	Value
Image pairs	1,500: official 1,000 trainval and 500 test
Image size	640 × 640 pixels
Annotations	10,013 bounding boxes across six defect classes
Classes	Open, short, mousebite, spur, copper, pin-hole
Prepared image format	Grayscale PNG; reference stays fixed
Groups	11 source groups; 332 test pairs from four groups absent in trainval

Source and labels

DeepPCB contains template/inspection pairs cropped from larger board scans and initially aligned by template matching. Each pair has a reference image, an inspected image, and a text label file with box corners and a class ID. Most annotated defects were added artificially by the dataset authors, so results may not transfer directly to real production defects. The extracted source tree is under `Data/DeepPCB-master/DeepPCB-master/PCBData`.

Partition	Pairs	Open	Short	Mousebite	Spur	Copper	Pin-hole
Trainval	1,000	1,283	1,028	1,379	1,142	1,010	1,031
Test	500	659	478	586	483	464	470
Total	1,500	1,942	1,506	1,965	1,625	1,474	1,501

Boxes range from 18–173 px wide (median 35) and 19–152 px high (median 33). Fifty-seven pairs contain at least one 3-channel JPEG; prepared outputs are converted to grayscale PNG.

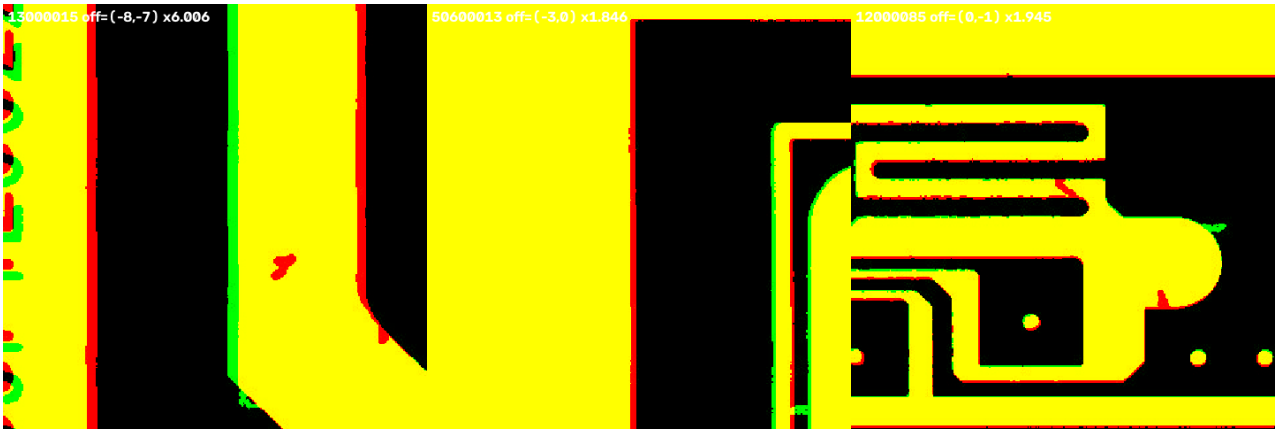
Preparation pipeline

The original data remain unchanged. The pipeline indexes and audits the source, creates a grouped validation split, generates seeded evaluation conditions, verifies image/label geometry, and exports COCO and YOLO data. Seed 6314 combined with the pair ID makes transforms independent of file order.

Scripts	Purpose
01–03: inspect, duplicate scan, alignment audit	Build source indexes; check files, candidate duplicates, and native offsets.
04: make splits	Create group-aware train/validation/test manifests and duplicate flags.
05 + pcb_shift	Create transformed image sets and boxes; provide training jitter utilities.
06: QA checks	Run geometry checks and generate visual check sheets.
07–09: COCO and YOLO export/check	Export model-ready annotations and images; verify the YOLO package.
run_all.sh	Run the complete preparation pipeline.

Data audit

Native alignment: an integer offset is estimated by minimizing template/inspection XOR disagreement over offsets up to ±8 px. A pair is flagged when the best non-zero offset reduces disagreement by at least 1.5×. This heuristic flags 123/1,500 pairs (75 train, 8 validation, 40 test), with offsets up to 8 px. Therefore, **s0 means "as supplied," not guaranteed perfectly aligned.**



Native misalignment examples. Red and green indicate pixels unique to each image; yellow indicates overlap.

Duplicates and data partitions

A difference-hash scan produced 337 candidates. Pixel-level comparison after shifts up to 8 px confirmed 40 duplicate pairs, all within the same source group. Three test pairs (50600065, 92000114, 92000118) duplicate training pairs and are flagged for a separate sensitivity analysis.

Grouped split

The official 500-pair test set is retained. One whole trainval group, group00041 (200 pairs), is held out for validation; the remaining 800 pairs form training data. This keeps transformed copies together. The validation group was selected by minimizing the L1 difference in class proportions among candidate group-held-out splits containing 17–23% of trainval.

Split	Pairs	Native-offset pairs	Group assignment
Train	800	75	13000, 20085, 44000, 50600, 77000, 92000
Validation	200	8	00041
Test	500	40	All 11 official test groups

Primary test results should include all 500 official test pairs. A clean-pair sensitivity analysis may exclude the 40 natively misaligned test pairs, leaving 460. The count 1,377 refers to clean pairs across the full dataset, not the test set.

The test set also contains 332 pairs from four groups absent from trainval. This provides an unseen-group stress test, while making the test group distribution different from training.

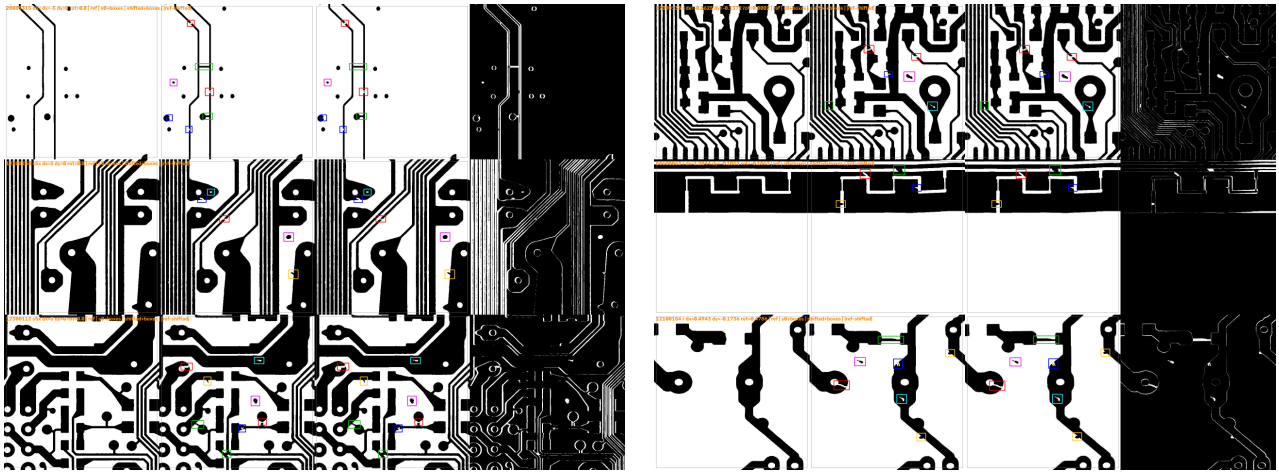
Controlled misalignment and labels

Validation and test each contain ten conditions: s0 (as supplied); horizontal and vertical shifts of 1, 2, 3, and 5 px; and a residual condition combining a 0.25–1.5 px translation with rotation between –0.5° and +0.5°. Training stores only s0; augmentation is generated separately.

For every condition, the reference stays unchanged and the same seeded transform is applied to the inspected image and its boxes. Integer shifts use exact pixel sampling; residual transforms use bilinear interpolation. For rotation, all four box corners are transformed and enclosed by a tight axis-aligned box.

Border and ignore handling

Evaluation excludes an 8 px border. Boxes within 14 px of an original image edge are marked ignore in every condition: 258/10,013 boxes (2.6%). Treat these as “don’t care” annotations during scoring.



Visual label checks for a 5 px shift (left) and residual transform (right).

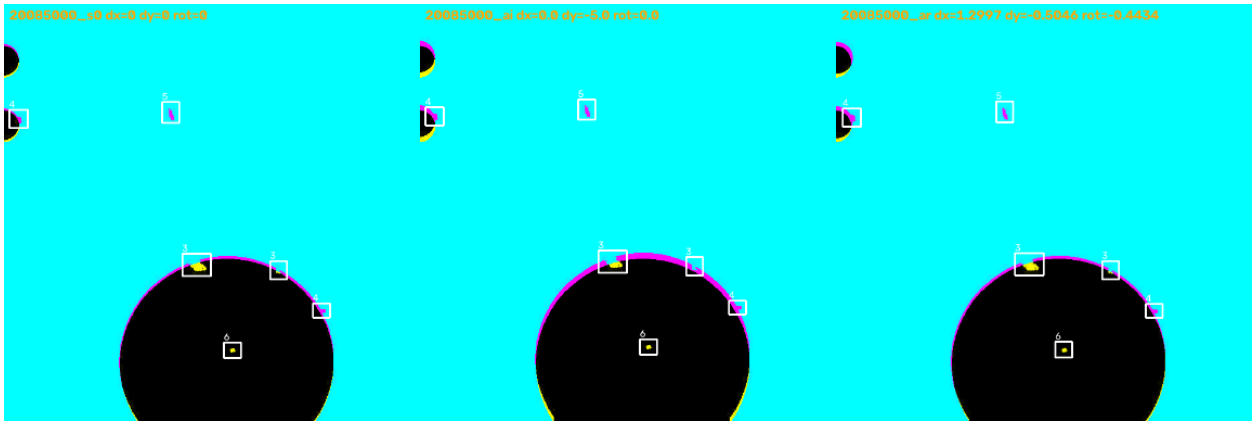
Exports and YOLO input

CSV manifests are the audit-friendly source of truth for pair IDs, groups, splits, transform parameters, paths, boxes, classes, and ignore flags. COCO JSON exports provide one file per available split/condition (21 files). The original DeepPCB text labels remain unchanged.

Artifact	Contents
sets/ref/	1,500 unchanged reference PNGs.
sets/{split}/{condition}/	7,800 grayscale inspection PNGs.
index/pairs.csv, boxes.csv, splits.csv	Source inventory, annotations, and partition metadata.
index/images.csv, labels.csv	Per-image transform metadata and per-box labels/ignore flags.
index/coco/	COCO exports for each available split/condition.
yolo_train.zip, yolo_eval_*.zip	YOLO training and evaluation composites with labels.
reports/, qa/	Machine-readable summaries and visual checks.

YOLOv10-Nano data path

YOLO inputs use three channels: reference, inspected image, and absolute difference. Labels use normalized `class xc yc w h` rows. The training package contains 800 unaugmented images and 2,400 augmented images (original, one random integer shift, and one residual transform per pair). Disable independent HSV channel augmentation because it corrupts the difference channel. YOLO labels cannot encode the ignore flag, so evaluation must apply the ignore rule from the master CSV.



Export check for original, shifted, and residual composites from one pair.

Verification and next steps

Automated check	Result
Integer-shift pixel checks	5,600 / 5,600 passed
Transformed box-content checks	36,640 / 36,640 passed
Non-ignored boxes inside valid region	41,220 / 41,220 passed
Residual transform audit (120 samples)	Median corner error 0.002 px; maximum 0.05 px
YOLO label presence, counts, and classes	10,200 / 10,200 passed
Composite channel construction (342 samples)	342 / 342 passed

The YOLO round-trip check reports coordinate differences for ignored boxes clipped at image boundaries. Evaluation should use the master annotations and apply the documented ignore rule. Rebuild the preparation outputs with Python 3, NumPy, OpenCV, and `bash MyData/scripts/run_all.sh`; set `PCB_ROOT` if the raw-data folder is elsewhere.

Limitations

Most defects are synthetic; 123 pairs have native alignment offsets; validation consists of one source group; and each validation/test pair has only one residual variant. Three train/test duplicate pairs also warrant a separately labeled sensitivity analysis.

Next steps

Implement registered and unregistered differencing baselines, train aligned-only and jitter-trained YOLO models, tune thresholds on validation at a common target recall, then evaluate all 500 official test pairs. Report the 460-pair clean-test subset and duplicate-excluded results only as sensitivity analyses. YOLOv10-Nano is the selected detector from the options listed in Proposal V4.

Reference

[1] S. Tang, F. He, X.-L. Huang, and J. Yang, “Online PCB defect detector on a new PCB defect dataset,” arXiv:1902.06197, 2019.